

```
In [ ]: #####
# Computer Vision and Image Processing Fundamentals
#
# Author: Justin Ogle
# #
Description:
# This project demonstrates fundamental computer vision and image
# processing techniques using Python, OpenCV, NumPy, TensorFlow/Keras, #
# and Matplotlib.
# # Topics
Covered:
# - Image Resizing and Preprocessing
# - Convolution Operations
# - Sobel and Prewitt Edge Detection
# - Deep Learning Image Classification
# - Image Enhancement
# - Laplacian Filtering
# - Gamma Correction
#
#####
```

```
#####
# Image Resizing and Preprocessing
#####
#Write the code resizing the Lena image to (224,224).
```

```
In [ ]:
```

```
In [2]: # Define Libraries
import cv2
from matplotlib import pyplot as plt

# Read the Lena image image =
cv2.imread('Lena.png')

# Resize the image to (224, 224)
resized_image = cv2.resize(image, (224, 224))

# Save the resized image
cv2.imwrite('resized_Lena.png', resized_image)

# Display Original Lena and Resized Lena images using matplotlib
plt.figure(figsize=(8, 4)) # Display Original Lena side
plt.subplot(1, 2, 1) plt.imshow(cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)) plt.title('Original Lena') # Display Resized
Lena image plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB))
plt.title('Resized Lena (224, 224)')
# Show images
plt.show()
```



```
#####
# Edge Detection Using Convolution Filters
# Sobel and Prewitt Operator Comparison
#####
```

In []: In [5]:

```
# Define Libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
I = cv2.imread('Lena.png')
# Recolor the image
G = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)

# Set a larger figure size
plt.figure(figsize=(12, 8))

# Display the original grayscale image
plt.subplot(2, 2, 1)
plt.imshow(G, cmap='gray')
plt.title('Original Grayscale Image')

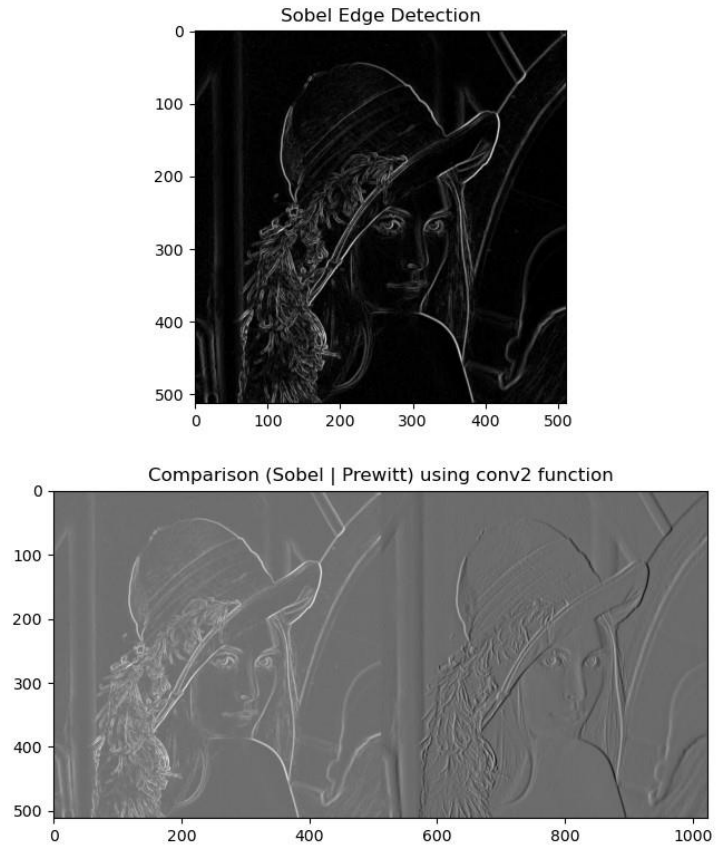
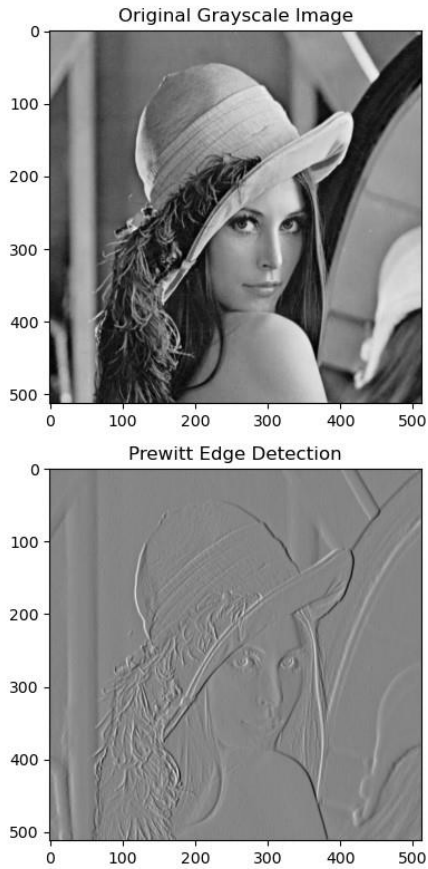
# Sobel edge detection using OpenCV
sobelx = cv2.Sobel(G, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(G, cv2.CV_64F, 0, 1, ksize=3)
CG = np.sqrt(sobelx**2 + sobely**2)

# Display the Sobel edge-detected image
plt.subplot(2, 2, 2)
plt.imshow(CG, cmap='gray')
plt.title('Sobel Edge Detection')

# Prewitt edge detection using OpenCV
kernel_x = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]])
CP = cv2.filter2D(G, cv2.CV_64F, kernel_x)

# Display the Prewitt edge-detected image
plt.subplot(2, 2, 3)
plt.imshow(CP, cmap='gray')
plt.title('Prewitt Edge Detection')
```

```
# Display the comparison between Sobel and Prewitt
plt.subplot(2, 2, 4) plt.imshow(np.hstack([CG, CP]),
cmap='gray') plt.title('Comparison (Sobel | Prewitt) using
conv2 function')
# Adjust layout for better spacing
plt.tight_layout() # Show images
plt.show()
```



```
#####
# Deep Learning Image Classification Using ResNet50
#####
#Classify Webcam Images Using Deep Learning.
#Classify some other objects to repeat the example and attach the screen copy of th
```

In []: In [17]:

```
# Define Libraries import cv2 from keras.applications.resnet50 import ResNet50,
preprocess_input, decode_predicti from keras.preprocessing import image import numpy
as np
from matplotlib import pyplot as plt

# Load pre-trained ResNet50 model model
= ResNet50(weights='imagenet')

# Create a video capture object for a Logitech webcam (adjust the device index if n
cap = cv2.VideoCapture(0) # Use 1 instead of 0 if the built-in webcam is 0

# Initialize variables to store predictions
```

```

predictions_list = []

# Capture and classify images in a loop for
i in range(100):
    # Capture a frame from the webcam
    ret, frame = cap.read()

    # Resize the image to the required input size for the network
    img = cv2.resize(frame, (224, 224))    img =
    image.img_to_array(img)    img = np.expand_dims(img, axis=0)
    img = preprocess_input(img)

    # Classify the image using the pre-trained ResNet50 model
    predictions = model.predict(img)
    decoded_predictions = decode_predictions(predictions)

    # Append the top predicted label and probability to the list
    predictions_list.append(decoded_predictions[0][0])

    # Display the image and the top predicted label
    plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.title(f"Predicted: {decoded_predictions[0][0][1]}")
    plt.axis('off')    plt.show()

    # Stop conditions either wait or if 'q' is pressed
    if cv2.waitKey(1) & 0xFF == ord('q'):        break
# Capture and classify images in a loop for
i in range(100):
    # Capture a frame from the webcam
    ret, frame = cap.read()

    # Check if the frame is empty (end of the video stream or camera not available)
    if not ret:        print("Error: Unable to capture frame.")        break

    # Resize the image to the required input size for the network
    img = cv2.resize(frame, (224, 224))    img =
    image.img_to_array(img)    img = np.expand_dims(img, axis=0)
    img = preprocess_input(img)

    # Classify the image using the pre-trained ResNet50 model
    predictions = model.predict(img)
    decoded_predictions = decode_predictions(predictions)

    # Append the top predicted label and probability to the list
    predictions_list.append(decoded_predictions[0][0])

    # Display the image and the top predicted label
    plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.title(f"Predicted: {decoded_predictions[0][0][1]}")    plt.axis('off')

```

```
plt.show()

# Stop conditions either wait or if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord('q'):          break

# Release the webcam cap.release()
cv2.destroyAllWindows()

# Create a bar plot for accumulated predictions
labels, probabilities = zip(*[(label[1], label[2]) for label in predictions_list])

plt.figure(figsize=(10, 5)) plt.bar(labels,
probabilities) plt.xlabel('Predicted Class')
plt.ylabel('Probability') plt.title('Accumulated
Predictions Over Frames') plt.xticks(rotation=45,
ha='right') plt.tight_layout() plt.show()
```

```
1/1 [=====] - 1s 1s/step
```

Predicted: bookcase

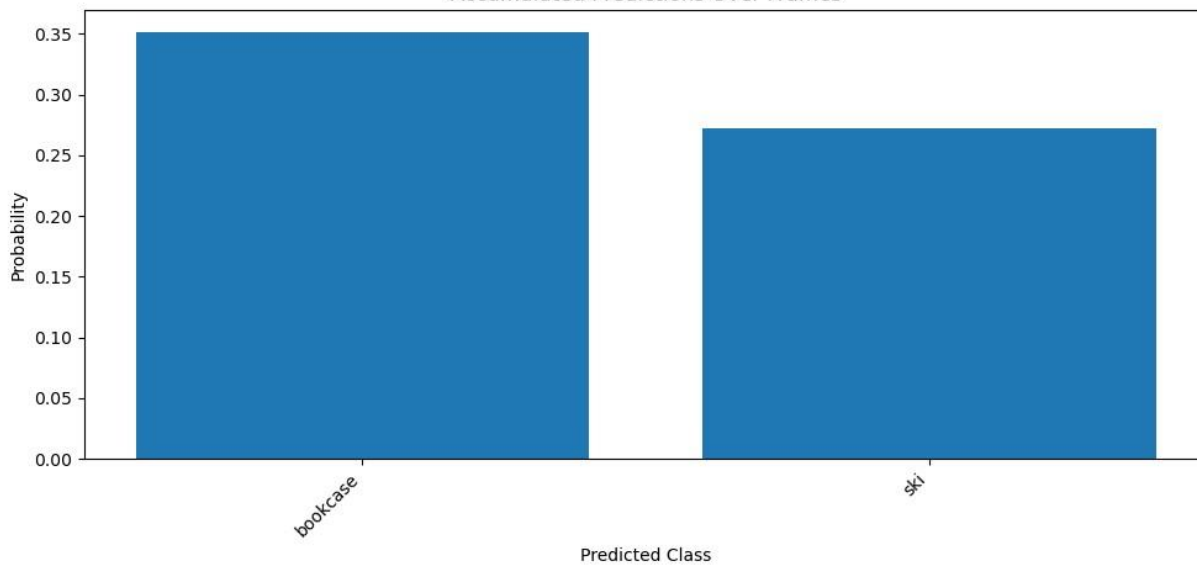


1/1 [=====] - 0s 114ms/step

Predicted: ski



Accumulated Predictions Over Frames



```
#####
# Image Enhancement and Feature Extraction Pipeline
#####
#Download the bone scan image from the website http://www.imageprocessingplace.com/
```

In []: In [13]:

```
# Define Libraries  
import cv2 import  
numpy as np  
from matplotlib import pyplot as plt  
  
# a.) Show Fig0343(a)(skeleton_orig).tif.  
# Path and define image location
```

```

image_path = 'Fig0343(a)(skeleton_orig).tif'
image_a = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
# Resize and plot image plt.figure(figsize=(16,
8))
plt.subplot(3, 3, 1), plt.imshow(image_a, cmap='gray'), plt.title('a.) Original Ima

# b.) Take the Laplacian of part a.) then show.
# Define and apply the Laplacian
laplacian_a = cv2.Laplacian(image_a, cv2.CV_64F)
# Plot the Laplacian and output stylings
plt.subplot(3, 3, 2), plt.imshow(laplacian_a, cmap='gray'), plt.title('b.) Laplacia

# c.) Sharpen the image of part a.) by adding the Laplacian then show.
# Define and apply the sharpening of the image sharpened_image =
cv2.addWeighted(image_a.astype(np.float64), 1.5, laplacian_a, -0.
# Plot the sharpened image and output stylings
plt.subplot(3, 3, 3), plt.imshow(sharpened_image.astype(np.uint8), cmap='gray'), pl

# d.) Take the Sobel of part a.) then show.
# Taking the Sobel of the original image
sobel_x = cv2.Sobel(image_a, cv2.CV_64F, 1, 0, ksize=3)
# Plotting the sobel with stylings
plt.subplot(3, 3, 4), plt.imshow(sobel_x, cmap='gray'), plt.title('d.) Sobel of Ima

# e.) Take the Sobel of part a.) and smooth with a 5x5 averaging filter then show.
# Repeat taking the Sobel and smothing the image with image resizing
sobel_y = cv2.Sobel(image_a, cv2.CV_64F, 0, 1, ksize=3) smoothed_sobel
= cv2.blur(sobel_y, (5, 5))
# Plotting the new smooth image
plt.subplot(3, 3, 5), plt.imshow(smoothed_sobel, cmap='gray'), plt.title('e.) Smoot

# f.) Mask the image formed by the product of part c.) and part e.) then show.
# Define the masked image and producing a product
masked_image = cv2.multiply(sharpened_image, smoothed_sobel.astype(np.float64))
# Plotting the Masked Image
plt.subplot(3, 3, 6), plt.imshow(masked_image.astype(np.uint8), cmap='gray'), plt.t

# g.) Sharpen image obtained by the sum of part a.) and part f.) then show.
# Define the sharpened final image and summing the parts
sharpened_final = cv2.addWeighted(image_a.astype(np.float64), 1, masked_image, 1, 0
# Plotting the Masked Image
plt.subplot(3, 3, 7), plt.imshow(sharpened_final.astype(np.uint8), cmap='gray'), pl

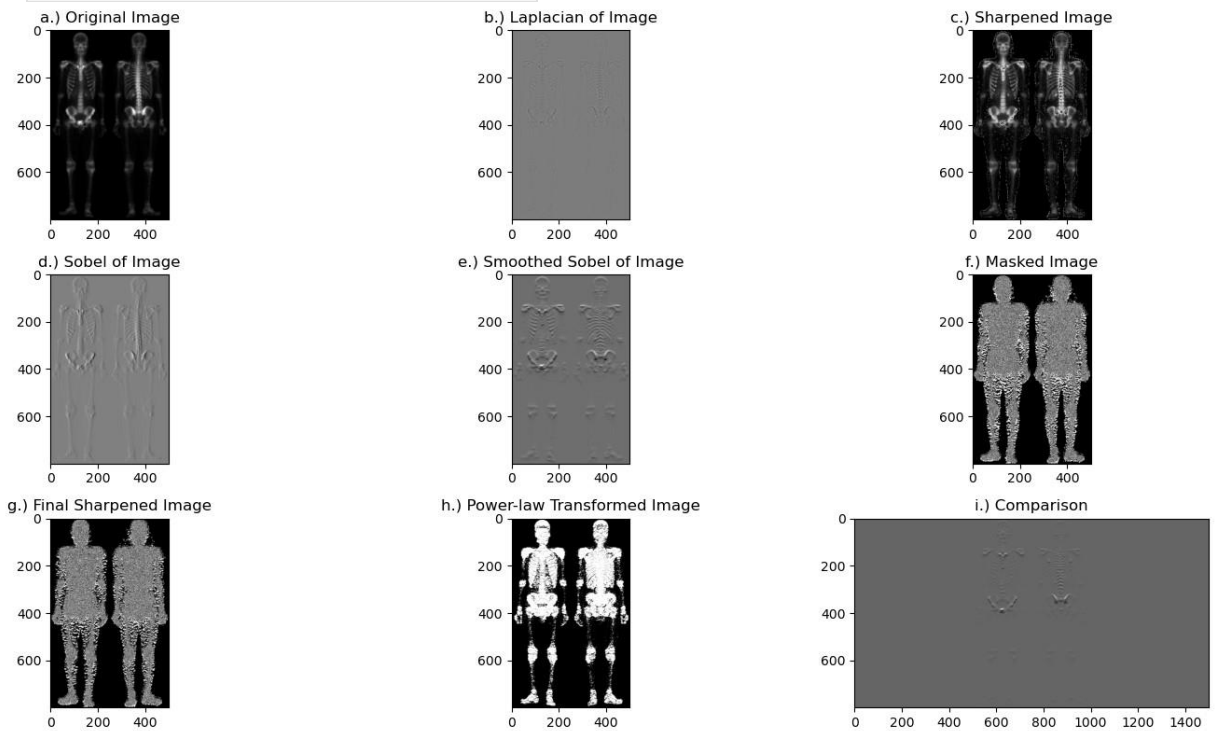
# h.) Final result obtained by applying power-law transformation to part g.) then s
gamma = 2.0 # Adjust the gamma value epsilon = 1e-5 # Small constant to avoid
division by zero power_law_transformed_image = np.power((sharpened_final + epsilon) /
255.0, gamma) power_law_transformed_image = np.clip(power_law_transformed_image, 0,
255) # Clip
# Plotting the Power-Law Transformed Image
plt.subplot(3, 3, 8), plt.imshow(power_law_transformed_image.astype(np.uint8), cmap

# i.) Compare part g.) and part h.) with part a.). plt.subplot(3, 3, 9),
plt.imshow(np.hstack([image_a, sharpened_final, power_law_tra

# Adjust layout for better spacing plt.tight_layout()

```

```
# Show images in a layout
plt.show()
```



```
In [ ]: #####
# Project Summary
#
# This project explores fundamental computer vision and image processing
# techniques through practical Python implementations. The workflow
# demonstrates image preprocessing, edge detection, image enhancement,
# feature extraction, and deep learning-based image classification using #
# pre-trained convolutional neural networks.
#
# The techniques implemented in this project form the foundation for
# modern computer vision systems used in robotics, autonomous systems, #
# image analytics, and artificial intelligence applications.
#
#####
```